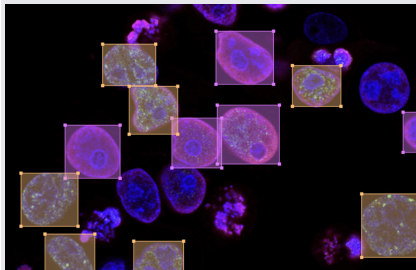
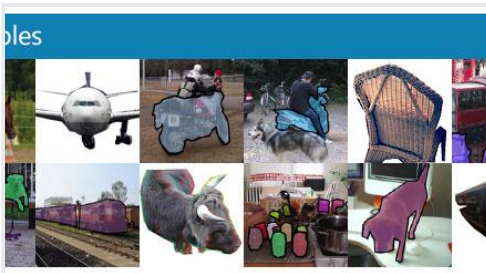


모델 성능을 결정하는 설계 자산

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9



오늘의 관점

데이터셋은 “모델의 학습 환경”이다

- 모델은 데이터셋에 담긴 패턴, 결측, 편향, 라벨 기준까지 함께 배웁니다.
- 같은 알고리즘이라도 데이터 분포가 바뀌면 성능과 실패 방식이 달라집니다.
- 따라서 데이터셋을 설계한다는 것은 문제 정의, 수집 기준, 라벨링 규칙, 평가 기준을 함께 설계하는 일입니다.

$$\hat{R}_{\mathcal{D}}(f) = \frac{1}{|\mathcal{D}|} \sum_{(x_i, y_i) \in \mathcal{D}} \ell(f(x_i), y_i)$$

모델 학습은 데이터셋 위에서 평균 손실을 줄이는 과정입니다.

데이터가 바뀌면 최적화되는 목표도 바뀝니다.

핵심 질문: “이 데이터셋으로 학습한 모델이 실제 사용 환경에서도 같은 문제를 풀 수 있는가?”

데이터셋의 기본 구성

샘플 하나는 세 층으로 읽는다

- **입력 (x)**: 이미지, 문장, 음성, 로그, 센서값처럼 모델이 관측하는 값
- **정답 (y)**: 클래스, 점수, 위치, 토큰 태그, 다음 행동처럼 모델이 맞춰야 하는 값
- **메타데이터 (m)**: 출처, 시간, 장비, 지역, 사용자군, 수집 조건처럼 데이터 해석에 필요한 맥락

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathcal{X}, y_i \in \mathcal{Y}$$



MNIST 예시 이미지. 출처: Wikimedia Commons, Josef Steppan, CC BY-SA 4.0.

Feature와 Label

모델이 보는 것과 맞춰야 하는 것

- **Feature (X)**: 모델에게 입력되는 설명 변수입니다. 집값 예측에서는 면적, 방 개수, 위치, 건축 연도 같은 값입니다.
- **Label 또는 Target (y)**: 모델이 맞춰야 하는 정답입니다. 집값 예측에서는 실제 거래가 또는 중간 주택 가격입니다.
- 지도학습 데이터셋은 보통 X와 y가 같은 행 순서로 정렬되어 있어야 합니다.

$$X \in \mathbb{R}^{n \times d}, \quad y \in \mathbb{R}^n \text{ or } \{1, \dots, C\}^n$$

행은 샘플, 열은 feature입니다.

라벨은 각 행에 대응되는 정답 벡터입니다.

데이터셋을 볼 때 첫 질문은 “무엇을 입력으로 보고, 무엇을 정답으로 예측하는가?”입니다.

집값 예측 예시

한 행(row)이 하나의 샘플이다

집값 예측 데이터에서 한 샘플은 보통 하나의 주택, 한 지역 블록, 또는 한 거래 기록입니다. 모델은 여러 feature를 보고 target인 가격을 예측합니다.

feature	의미
area	전용면적 또는 대지면적
rooms	방 개수
age	건축 후 경과 연수
distance	도심 또는 역까지 거리
school_score	학군 또는 주변 환경 지표

예측 목표

구분	값의 예
입력 (x_i)	면적 84㎡, 방 3개, 12년, 역 0.7km
정답 (y_i)	실거래가 7.2억 원
모델 출력 (_i)	예측 가격 7.0억 원

$$RMSE = \sqrt{\frac{1}{m} \sum_{j=1}^m (\hat{y}_j - y_j)^2}$$

집값 예측은 보통 연속값을 맞추는 회귀(regression) 문제입니다.

실제 유명 예시: California Housing

scikit-learn의 회귀 데이터셋

- California Housing은 집값 예측을 설명할 때 자주 쓰는 공개 회귀 데이터셋입니다.
- scikit-learn 문서 기준으로 20,640개 샘플과 8개 feature를 갖습니다.
- target은 중간 주택 가격이며, 단위는 100,000 [?]입니다.
- feature는 평균 소득, 주택 평균 연령, 평균 방 수, 평균 침실 수, 인구, 평균 거주 인원, 위도, 경도입니다.

데이터셋 구조로 읽기

요소	California Housing에서의 의미
sample	한 지역 블록의 관측값
feature	MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude
target	MedHouseVal

문제 유형별 Label의 의미

label은 문제에 따라 모양이 달라진다

- 분류 문제에서는 label이 클래스 번호나 클래스 이름입니다.
- 회귀 문제에서는 label이 연속적인 수치입니다.
- 객체 탐지에서는 label이 클래스와 bounding box를 함께 가집니다.
- 생성/캡셔닝에서는 label이 정답 문장 또는 다음 토큰 시퀀스가 됩니다.

문제	입력 (x)	label (y)
손글씨 분류	숫자 이미지	0-9
집값 예측	주택 feature	가격
객체 탐지	사진	클래스 + 박스 좌표
감성 분석	리뷰 문장	긍정/부정
이미지 캡셔닝	사진	설명 문장

같은 데이터라도 label 정의가 바뀌면 완전히 다른 학습 문제가 됩니다.

Feature 설계에서 자주 생기는 문제

feature는 “모델이 볼 수 있는 세계”다

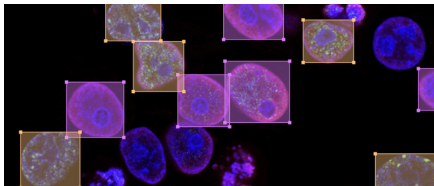
- 좋은 feature는 예측 대상과 관련이 있고, 배포 시점에도 안정적으로 얻을 수 있어야 합니다.
- 학습 시점에는 알 수 있지만 예측 시점에는 알 수 없는 feature는 사용하면 안 됩니다.
- 결측치, 범주형 값, 단위 차이, 극단값은 모델 종류에 따라 성능에 큰 영향을 줍니다.

문제	집값 예측 예
단위 불일치	평, m ² , 달러, 원이 섞임
미래 정보	거래 후 확정되는 세금/등기 정보 사용
범주형 처리	지역명, 건물 유형, 학군
결측치	방 개수, 연식, 위치 정보 누락
이상값	0원 거래, 비정상 면적, 잘못된 좌표

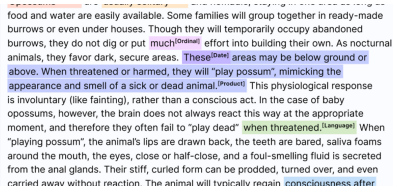
feature는 많을수록 좋은 것이 아니라, 예측 시점에 사용할 수 있고 의미가 안정적인 것이 중요합니다.

데이터 유형별로 달라지는 관리 포인트

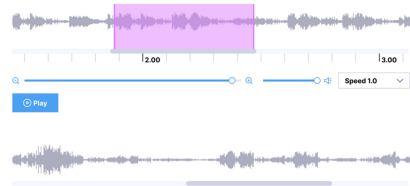
유형	예시	라벨/정제의 핵심
표 데이터	고객 정보, 진단 수치, 거래 로그	단위, 결측치, 이상값, 집계 기준
이미지·영상	제품 사진, 의료 영상, CCTV	해상도, 촬영 조건, 객체 경계, 개인 정보
텍스트	리뷰, 뉴스, 상담 기록	토큰화, 중복, 저작권, 표현 편향
음성·시계열	음성 명령, 센서 로그	샘플링 주기, 잡음, 시간 정렬



이미지 라벨링 예시. 출처: Label Studio.



텍스트 라벨링 예시. 출처: Label Studio.



음성 라벨링 예시. 출처: Label Studio.

사례 1: MNIST



MNIST test sample. 출처: Wikimedia Commons, Josef Steppan, CC BY-SA 4.0.

잘 정의된 작은 벤치마크

- 손글씨 숫자 0-9를 분류하는 대표적인 이미지 분류 데이터셋입니다.
- TensorFlow Datasets 기준으로 학습 60,000개, 테스트 10,000개가 제공됩니다.
- 각 이미지는 (28) 형태의 회색조 이미지입니다.
- 장점: 작고 단순하여 모델 구조와 학습 파이프라인을 빠르게 검증하기 좋습니다.
- 한계: 실제 카메라 이미지, 배경 변화, 왜곡, 새로운 필기 습관까지 대표하지는 않습니다.

사실 출처: TensorFlow Datasets MNIST catalog.

사례 2: MS COCO

“맥락 속 객체”를 다루는 데이터셋

- COCO는 객체 탐지, 세그멘테이션, 캡셔닝에 널리 쓰이는 대규모 이미지 데이터셋입니다.
- 공식 설명 기준으로 330K 이미지, 200K+ 라벨 이미지, 1.5M 객체 인스턴스, 80 객체 카테고리, 이미지당 5개 캡션을 포함합니다.
- 같은 “사람” 또는 “자동차”라도 배경, 가림, 크기, 상호작용이 다양하기 때문에 단순한 아이콘 이미지보다 실제 환경에 가깝습니다.
- COCO 같은 데이터셋은 모델 성능뿐 아니라 평가 서버, 리더보드, 데이터 포맷까지 생태계를 형성합니다.

Dataset examples



COCO examples. 출처: cocodataset.org / COCO GitHub.



데이터 수집 설계

먼저 정해야 하는 것

- **모집단**: 모델이 실제로 만날 대상은 누구/무엇인가?
- **표본추출**: 무작위인가, 편의표본인가, 특정 조건을 의도적으로 더 모았는가?
- **수집 기간**: 계절, 이벤트, 정책 변화, 장비 교체가 반영되었는가?
- **수집 단위**: 한 사용자, 한 문서, 한 장소, 한 장면을 어떻게 묶을 것인가?
- **제외 기준**: 품질 낮은 데이터, 민감 정보, 중복 표본을 어떻게 처리할 것인가?

설계가 약할 때 생기는 문제

문제	결과
쉬운 샘플 위주 수집	실제 배포 후 난이도 상승
특정 집단 과소대표	집단별 성능 격차
같은 사용자 반복 포함	테스트 성능 과대평가
오래된 데이터 유지	운영 환경과 분포 불일치

수집 기준은 실험의 “전제 조건”입니다. 나중에 모델만 바뀌어서는 전제 오류를 해결하기 어렵습니다.

라벨링 설계

라벨 품질은 운영 프로세스다

- 라벨 정의는 “정답 목록”이 아니라 경계 사례를 다루는 의사결정 문서입니다.
- 라벨러 교육, 예시집, 금지 사례, 검수 기준이 함께 있어야 합니다.
- 라벨러 간 불일치는 오류라기보다 기준이 모호하다는 신호일 수 있습니다.
- 처음부터 전량 라벨링하기보다 샘플 라벨링 → 일치도 측정 → 기준 수정 → 본 라벨링 순서가 안정적입니다.

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

Person^[1] Fact^[2] Date^[3] Time^[4] Ordinal^[5] Product^[6] Language^[7] Location^[8]

Opossums^(Person) are usually solitary^(Fact) and nomadic, staying in one area as long as food and water are easily available. Some families will group together in ready-made burrows or even under houses. Though they will temporarily occupy abandoned burrows, they do not dig or put much^(Ordinal) effort into building their own. As nocturnal animals, they favor dark, secure areas. These^(Date) areas may be below ground or above. When threatened or harmed, they will “play possum”, mimicking the appearance and smell of a sick or dead animal^(Product). This physiological response is involuntary (like fainting), rather than a conscious act. In the case of baby opossums, however, the brain does not always react this way at the appropriate moment, and therefore they often fail to “play dead” when threatened.^(Language) When “playing possum”, the animal’s lips are drawn back, the teeth are bared, saliva foams around the mouth, the eyes, close or half-close, and a foul-smelling fluid is secreted from the anal glands. Their stiff, curled form can be prodded, turned over, and even carried away without reaction. The animal will typically regain consciousness after a period of between 40 minutes and 4 hours, a process^(Time) which begins^(Location) with a slight twitching of the ears.

텍스트 엔티티 라벨링 화면. 출처: Label Studio.

학습·검증·테스트 분리

왜 세 개가 필요한가

- 학습 세트만 있으면 모델이 외운 것인지 일반화한 것인지 구분하기 어렵습니다.
- 검증 세트는 모델을 고르는 과정에 쓰입니다.
- 테스트 세트는 선택이 끝난 뒤 “최종 보고”에만 써야 합니다.
- 세 분할을 섞어 쓰면 성능 숫자가 실제보다 좋아 보입니다.

$$\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset, \quad \hat{R}_{\text{test}}(f) = \frac{1}{m} \sum_{j=1}^m \ell(f(x_j), y_j)$$

역할을 분리해서 읽기

분할	하는 일
Train	파라미터를 학습
Validation	모델 구조, 하이퍼파라미터, 임계값 선택
Test	선택된 모델의 마지막 성능 추정

검증 세트에서 여러 번 고른 모델은 이미 검증 세트에 맞춰진 모델입니다.

Train Set

모델이 직접 배우는 데이터

- 모델의 가중치나 파라미터는 train set의 손실을 줄이는 방향으로 업데이트됩니다.
- 데이터 증강, 샘플 가중치, 클래스 재균형 같은 학습 전략은 보통 train set에 적용합니다.
- train 성능이 좋다는 것은 “학습 데이터에 잘 맞는다”는 뜻이지, 곧바로 실전 성능을 뜻하지 않습니다.

집값 예측에서의 train set

항목	예
입력	과거 주택 feature
정답	과거 실거래가 또는 중간 주택 가격
학습	가격 오차가 작아지도록 회귀 모델 업데이트
주의	같은 단지/같은 거래 묶음이 다른 분할에 새지 않게 관리

Validation Set

모델을 고르는 데이터

- 검증 세트는 모델 후보를 비교하는 데 사용합니다.
- 예: 학습률, 트리 깊이, 정규화 강도, feature 선택, threshold를 고릅니다.
- 검증 점수를 보고 여러 번 수정했다면, 그 검증 세트도 의사결정에 사용된 것입니다.

$$\lambda^* = \arg \min_{\lambda \in \Lambda} \hat{R}_{\text{val}}(f_{\lambda}), \quad \hat{R}_{\text{test}}(f_{\lambda^*}) \text{ is reported once}$$

검증 세트로 선택한 뒤, 테스트 세트 성능은 마지막에 한 번만 보고합니다.

Validation은 개발 중 의사결정을 위한 데이터이고, Test는 최종 보고를 위한 데이터입니다.

Test Set

마지막까지 숨겨 두는 데이터

- 테스트 세트는 모델 선택, 튜닝, 전처리 결정에 사용하면 안 됩니다.
- 테스트 결과를 본 뒤 모델을 고치면, 그 테스트 세트는 더 이상 순수한 테스트 세트가 아닙니다.
- 논문, 리더보드, 과제 보고서에서 가장 중요한 숫자는 보통 test 성능입니다.

테스트 세트가 답하는 질문

질문	의미
처음 보는 샘플에서도 맞 는가?	일반화 성능
특정 집단에서도 맞는가?	공정성/대표성
rare case에서도 버티는 가?	안정성
운영 비용 기준을 만족하 는가?	실용성

분할 방법을 고르는 기준

무작위 분할이 항상 정답은 아니다

- 독립적인 샘플이 충분히 섞여 있다면 random split이 간단합니다.
- 클래스 비율이 중요한 분류 문제는 stratified split이 더 안정적입니다.
- 같은 사용자, 같은 장소, 같은 문서 묶음이 있으면 group split이 필요합니다.
- 시간 순서가 중요한 문제는 과거로 학습하고 미래로 평가해야 합니다.

상황	권장 분할
일반 표 데이터	random split
클래스 불균형 분류	stratified split
사용자/환자/문서 단위 반복	group split
주가, 로그, 센서 시계열	time-based split
지역 일반화 확인	region holdout

scikit-learn `train_test_split`은 `test_size`, `train_size`, `random_state`, `shuffle`, `stratify` 옵션을 제공합니다.

데이터 누수(leakage)

누수는 가장 흔한 평가 실패다

데이터 누수는 학습 과정에서 알면 안 되는 정보가 모델이나 전처리 과정에 들어가는 현상입니다. 누수가 있으면 테스트 성능은 좋아 보이지만, 배포 후 성능이 급격히 떨어질 수 있습니다.

누수 유형	왜 위험한가
같은 사용자의 기록이 train/test에 동시에 존재	사용자 고유 패턴을 외움
원본 문서와 요약문이 다른 분할에 존재	내용 중복으로 평가 붕괴
시계열 데이터를 무작위로 섞음	미래 정보가 학습에 들어감
전처리를 전체 데이터에 먼저 fit	테스트 통계가 학습에 반영됨

분할 기준은 행(row)이 아니라 “정보가 공유되는 단위”를 기준으로 잡아야 합니다.

데이터 품질 점검

품질을 보는 네 축

- **정확성**: 입력값과 라벨이 실제 현상과 일치하는가?
- **완전성**: 필수 필드, 라벨, 메타데이터가 빠지지 않았는가?
- **일관성**: 동일한 규칙이 기간/라벨러/출처에 걸쳐 유지되는가?
- **대표성**: 실제 운영 분포와 충분히 비슷한가?

$$\Delta_{\text{gen}} = \left| \hat{R}_{\text{test}}(f) - \hat{R}_{\text{train}}(f) \right|$$

체크 방법

점검	예시
중복	이미지 perceptual hash, 텍스트 near-duplicate
결측	필드별 결측률, 라벨 누락률
이상값	단위 오류, 불가능한 날짜, 비정상 범위
라벨 오류	샘플 재검수, 다수결, gold set 비교
분포 차이	학습/테스트/운영의 피쳐 분포 비교

불균형과 편향

불균형은 성능 평균을 속인다

- 다수 클래스가 대부분이면 전체 정확도는 높아 보어도 소수 클래스는 거의 맞추지 못할 수 있습니다.
- 의료, 금융, 안전 분야에서는 소수 클래스가 오히려 가장 중요한 오류일 수 있습니다.
- 클래스별 지표, 집단별 지표, 비용 기반 지표를 함께 보고해야 합니다.

$$\pi_c = \frac{n_c}{n}, \quad w_c \propto \frac{1}{n_c + \epsilon}$$

$$\mathcal{L}_{\text{weighted}} = - \sum_{c=1}^C w_c y_c \log \hat{p}_c$$

편향 점검 질문

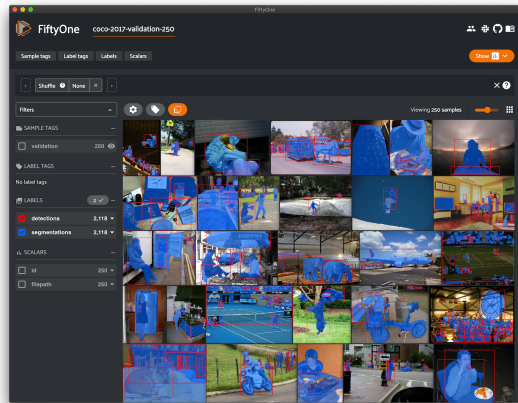
질문	확인할 것
누가 과소대표되는가?	지역, 연령, 언어, 장비, 상황
어떤 라벨이 모호한가?	주관 판단, 경계 사례, 문화 차이
어떤 오류가 더 큰 피해를 만드는가?	false positive와 false negative의 비용
데이터 사용 권한은 명확한가?	저작권, 개인정보, 민감정보, 동의

평가 지표 설계

지표는 목적 함수와 다르다

- 학습 손실은 모델을 업데이트하기 위한 수학적 신호입니다.
- 평가 지표는 실제 문제에서 무엇을 잘했다고 볼지 정하는 기준입니다.
- 정확도만으로는 클래스 불균형, 비용 차이, 안전 요구사항을 충분히 설명하지 못합니다.
- 따라서 전체 평균, 클래스별 평균, 집단별 성능, 운영 임계값을 함께 봐야 합니다.

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$



COCO 데이터 탐색/평가 예시. 출처: COCO GitHub / FiftyOne 이미지.

데이터셋 문서화

Datasheets for Datasets

Geburu et al.은 데이터셋에도 전자 부품의 datasheet처럼 생성 목적, 구성, 수집 과정, 권장 사용, 한계를 문서화해야 한다고 제안했습니다.

데이터셋 문서화는 단순한 행정 작업이 아니라 재현성, 책임성, 유지보수성을 위한 기술 부채 관리입니다.

항목	남겨야 할 내용
Motivation	왜 만들었고 어떤 문제를 위한 것인가
Composition	어떤 샘플, 라벨, 메타데이터가 있는가
Collection	누가, 언제, 어떤 조건에서 모았는가
Preprocessing	정제, 필터링, 익명화 기준은 무엇인가
Uses	권장/비권장 사용 범위는 무엇인가
Maintenance	

운영 중 데이터 드리프트

배포 후에도 데이터셋은 변한다

- 사용자 행동, 기기, 계절, 정책, 인터페이스가 바뀌면 입력 분포가 바뀝니다.
- 라벨 기준이 바뀌거나 새로운 클래스가 등장하면 정답 분포도 바뀝니다.
- 운영 성능 하락은 모델 구조 문제가 아니라 데이터 생성 과정 변화에서 시작되는 경우가 많습니다.

$$\text{PSI} = \sum_{k=1}^K (p_k - q_k) \log \frac{p_k}{q_k}$$

모니터링 항목

항목	예시
입력 분포	주요 피처의 평균, 분위수, 결측률
라벨 분포	클래스 비율, 신규 라벨, rare case 증가
성능 분포	집단별 recall, false negative rate
데이터 파이프라인	스키마 변경, 수집 실패, 중복 증가

핵심 요약

1. 정의

문제, 입력, 정답, 성공 지표를 먼저 고정합니다.

2. 수집

실제 운영 분포와 표본추출 기준을 문서화합니다.

3. 라벨링

라벨 기준, 경계 사례, 검수 루프를 운영합니다.

4. 분할

정보 누수가 생기지 않는 단위로 train/val/test를 나눕니다.

5. 평가

전체 평균뿐 아니라 클래스별·집단별 오류를 봅니다.

6. 유지보수

문서화, 버전 관리, 드리프트 모니터링을 계속합니다.

좋은 AI 모델을 만들려면 먼저 좋은 데이터셋을 정의하고, 측정하고, 관리해야 합니다.

참고자료 및 이미지 출처

- TensorFlow Datasets, MNIST catalog: <https://www.tensorflow.org/datasets/catalog/mnist>
- scikit-learn, fetch_california_housing: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html
- scikit-learn, train_test_split: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
- Wikimedia Commons, File:MnistExamples.png, Josef Steppan, CC BY-SA 4.0: <https://commons.wikimedia.org/wiki/File:MnistExamples.png>
- COCO official website / GitHub assets: <https://cocodataset.org/>, <https://github.com/cocodataset/cocodataset.github.io>
- COCO dataset overview facts: 330K images, 200K+ labeled images, 1.5M object instances, 80 categories, 5 captions per image
- Label Studio official website screenshots: <https://labelstud.io/>
- Gebru et al., "Datasheets for Datasets", arXiv:1803.09010: <https://arxiv.org/abs/1803.09010>

수식 SVG는 `assets/latex/`의 LaTeX 원본에서 생성했습니다.